ISEC LAB · METHODS REPORT

Find Forms: how the crawler works and how its keywords were chosen

A Selenium crawler that visits websites like a first-time visitor, finds their email newsletter signup forms, and records each one with screenshots and design measurements for manual coding.

Hrishikesh Jadhav Northeastern University Method as of 21 October 2026

CONTENTS

1. What the crawler does
2. How one site is crawled
3. How one page is scanned
4. How a form is recognised
5. What is recorded
6. How the keywords were collected
7. Differences from the original method
8. Limitations
9. Planned work: submitting the forms

1. What the crawler does

The study looks at how websites design their email subscription forms: footer signups, popups, slide-ins and sticky bars. For each site in a list, the crawler opens the site in a real Chromium browser, driven by Selenium, waits, scrolls and moves the mouse the way a visitor would, detects every signup form that appears, and saves what it found.

The detection logic is adapted from Englehardt, Han and Narayanan (2018), who built a crawler to sign up for newsletters at scale [1]. Our goal is different: we catalogue the forms instead of submitting them, so several parts of their method were changed (section 7).

SAFEGUARDS

- **Detection only.** The crawler never types into, fills, submits or clicks anything. It moves between pages by opening link addresses directly.
- **Polite pacing.** One browser visits one site at a time, with a 3-second pause plus 0–1 s of random jitter between page loads, and at most 3 pages per site by default.
- **Real browser only.** Every request comes from ordinary Chromium page loads; nothing is fetched outside the browser.
- **robots.txt is not consulted.** This was a deliberate project decision (8 Oct 2026) and should be stated in the methods section.

Larger lists can be split across up to 8 browsers running in separate processes (`--jobs`); their results are merged afterwards. A run can be stopped and resumed at any time: finished sites are logged and skipped on the next run.

---

2. How one site is crawled

1. **Fresh browser profile.** Each site gets its own browser: a 1440 × 900 desktop window, US-English locale, no cookies or storage left over from earlier sites. Video and audio are not downloaded; images, fonts and third-party scripts load normally, because many signup forms are embedded from other services.
2. **Landing page.** The homepage is scanned for forms (section 3).
3. **Follow links if needed.** If the homepage has no signup form, the crawler ranks the visible links on the page by how likely they are to lead to one and visits the best ones in order, stopping as soon as a page yields a form, or after 3 pages in total.
4. **Time limit.** If one site is still running after 6 minutes (2 minutes per allowed page), its browser is stopped, the site is logged as timed out, and the crawl moves on with a fresh browser. This guards against pages whose scripts never let the browser respond; in the pilot, two such sites had each held a browser for 30 minutes and almost 5 hours.

Link ranking

A link is checked against five groups in order and gets the first group it matches. Lower groups are visited first; links matching none are not visited. Links whose text mentions *unsubscribe*, *mobile* or *phone*, links whose address contains *unsubscribe*, and file downloads are always skipped.

Group	Checks the link's	Matches if it contains	Source
1	text	newsletter, weekly ad, subscri..., inbox, email, sale alert	paper, “subscribe” shortened
2	text	signup, sign up, sign me up, register, create, join, suscr..., inscr..., count me in, send me	paper + keyword study
3	address	newsletter, subscri..., signup, sign-up, mailing-list	keyword study
4	address	/article, news/, a year such as /2024/	paper
5	address	/us/, =us&, en-us	paper

Matching ignores capitalisation and counts a keyword anywhere in the text or address. “...” marks a word stem: *subscri* matches subscribe, subscriber and subscription.

3. How one page is scanned

Many signup forms are not on the page when it first loads: they open after a delay, after the visitor scrolls, or when the mouse moves toward the top of the window as if to leave (*exit intent*). The crawler therefore checks the page four times, once after each of these triggers, and records the first moment each form appeared.



One page scan, drawn to scale. Forms are detected after each step; each form is recorded once, under the first check that saw it.

Check	What happens first	Catches	Pilot forms
`on_load`	page load, then 1 s for instant popups	inline, footer and header forms; instant popups	64
`after_delay`	8 s wait	timed popups	10
`after_scroll`	scroll down in steps to the bottom, then back up	scroll-triggered popups, bottom sticky bars	5
`after_exit_intent`	mouse moved to the top edge; “pointer left the window” events	exit-intent popups	1

Pilot: 200 sites crawled on 11 October 2026, 80 forms recorded. One form in five (16 of 80) appeared only after a trigger.

Each check runs in every frame of the page, so forms inside embedded iframes are found as well.

4. How a form is recognised

Detection runs as a script inside the page. It uses two complementary approaches from the original paper: *top-down*, which starts from `<form>` elements, and *bottom-up*, which starts from an email box and looks for its button. The second catches the many modern forms built without a `<form>` tag.

STEP 1 · WHERE TO LOOK

The page plus every open **shadow DOM** (sealed-off widget areas that a normal page search does not enter), up to 15,000 elements.

↓

STEP 2 · IS THIS AN EMAIL BOX?

A visible `type="email"` input, or a text input whose name, placeholder or label mentions *email* or *newsletter* and not *search*, *password*, *phone*, *coupon* or similar.

↓

STEP 3 · TOP-DOWN

Every `<form>` that contains a visible email box is a candidate.

STEP 4 · BOTTOM-UP

For an email box outside any `<form>`: climb up the page structure, at most 15 levels, to the first element that also contains a visible **submit-like control**. That element is the candidate.

↓

STEP 5 · RANK

(1) furthest in front first (highest z-index), so popups come before inline forms; ties go to popup-looking forms; (2) login-like forms last; (3) forms with more fields first.

↓

STEP 6 · MEASURE

Position, fields, labels, button and surrounding text, size; then an element screenshot, a full-window screenshot and the form's HTML are saved.

Detection in one frame. Every candidate is recorded, not only the best one.

Submit-like controls

Any `<button>` (except reset) and any submit or image input counts. A short clickable `<div>`, `<span>` or link counts only if its text, at most 40 characters, contains a submit word: *subscribe*, *sign up*, *sign me up*, *join*, *submit*, *send*, *notify*, *register*, *get*, *continue*, *next*, *save*, *count me in*, *yes*, *go*, *ok*, or the Spanish and Portuguese/French stems *suscr…* and *inscr…*.

Login-like forms

Login forms that ask for an email address look like signups, so they are flagged rather than dropped. A form is marked `login_like` if any of these hold:

- it contains a password box;
- its HTML contains *log in*, *login*, *sign in* or *signin* as a word;
- the plain text around it, not counting links and buttons, contains those words (for example “Log in to Hulu with your MyDisney account” above an email box);
- the page address looks like a login or account page (`auth.`, `/login`, `/signin`, `/account`). Two-step logins show only an email box and “Continue”; the password box would appear on the next screen, which the crawler never opens.

The word-boundary rule was added after a pilot check showed that the original text match fired inside other words: 3 of the 5 forms flagged in an 18-form pilot were newsletter forms caught by “By signing up, I agree…”.

---

5. What is recorded

Each unique form on a site becomes one row (37 columns) in `forms.csv`, with a matching JSON line, an element screenshot, a screenshot of the whole window, and the form's HTML. Every page visit is logged in `pages.csv`, and every run in a timestamped `run.log` that ends with the total run time.

Group	Columns
Where and when	site, page and frame address, first check that saw it, detection method, rank on the page
Position	`placement_guess` (fullscreen takeover, sticky bar top/bottom, modal popup, slide-in, footer, header, sidebar, in article, inline, near page bottom), z-index, popup hint, in iframe, in shadow DOM, width, height, share of the window covered
Fields	number and types of fields, field names, checkboxes and how many are pre-checked, fields with no label
Email box	input type, whether it has a label, placeholder, autocomplete, required
Words	submit text, other clickable text near the form (e.g. “No thanks”), heading, consent wording, form text
Files	element screenshot, window screenshot, saved HTML

site	https://arlingtonpower.com
first_seen_phase	after_delay
detection_method	top-down
placement_guess	modal_popup
field_types	email | checkbox | checkbox
n_checkboxes_prechecked	0
email_placeholder	Email
submit_text	Submit
other_clickable_text	Close dialog
viewport_coverage	0.325

Part of one real row from the pilot: a popup that opened during the 8-second wait and covered about a third of the window.

Columns such as `placement_guess`, `login_like`, `modal_hint`, `consent_text_hint` and `heading` are heuristics. They are starting points for manual coding and are checked against the screenshots.

6. How the keywords were collected

The paper's keyword lists decide two things: which links the crawler follows when the homepage has no form (section 2), and which custom buttons count as submit buttons (section 4). On 11 October 2026 we tested and extended these lists against real signup pages.

6.1 Data

We assembled a list of **146 URLs**, from 144 sites, of pages known to contain an email subscription form (news sites, shops, Substack newsletters, company blogs and others). A separate helper script loaded each page in 5 parallel browsers, found the form with the crawler's own detection code, and recorded the page title, main heading, address path, and the form's heading, button text, email placeholder and surrounding text. As in the crawler, nothing was clicked or typed.

Result	Pages
form found	118
page loaded, no form detected	22
failed to load	3
HTTP error	3
total	**146**

159 forms were found on the 118 pages.

6.2 Analysis

Button text. The 115 pages with button text held 127 buttons. After ignoring capitalisation, spacing and symbols (“SIGN UP”, “Sign Up >”), they reduce to 30 distinct wordings.

Buttons	Wording	Already matched?
62	subscribe	yes, group 1
19	submit	no
17	sign up	yes, group 2
2	suscribirse (Spanish)	no
2	next	no
1 each	send me the next briefing, count me in, inscreva-se (Portuguese), update my subscriptions, save newsletter options, …	mostly no

Form text. The text of the 108 forms was split into single words and two-word phrases and counted once per form: 3,091 terms, 363 of them used by 3 or more forms. Most frequent were field labels (*first name*, *address*) and legal text (*privacy policy*, *terms*). Terms describing the newsletter itself included *news* (25 forms), *updates* (15), *weekly* (14), *latest* (14), *daily* (12), and *delivered*, *free*, *offers*, *events* and *stay* (10 each).

Page addresses. 113 of the 146 URLs had a path (the rest were homepages). The most common path words were *newsletter* (40 pages), *signup* (27), *subscribe* (24), *newsletters* (20) and *sign-up* (8). The paper looks for signup words only in a link's text, never its address, so a link reading “Stay informed” that points to `/newsletter` was never followed.

6.3 What was added

List	Change	Evidence
Link group 1	changed “subscribe” → stem “subscri”	“update my subscriptions” went unmatched: *subscribe* is not contained in *subscription*
Link group 2	added suscr, inscr, count me in, send me	signup buttons in Spanish (2 forms), Portuguese (1), and signup phrasing (1 each). *suscr* rather than *suscri*, so the accented *suscríbete* also matches
Link group 3	new address contains newsletter, subscri, signup, sign-up, mailing-list	catches 101 of the 113 signup URLs with a path; links to */unsubscribe* are explicitly excluded
Submit words	added suscr..., inscr..., next, save	button wordings the detector did not yet recognise

Button words went into group 2 rather than group 1: a link reading “Sign up” or “Count me in” often leads to a signup page, but also to account registration, so it is a weaker signal than *newsletter* or *subscribe*.

6.4 What was considered and not added

Because a link keyword matches anywhere in a link's text, each candidate was judged by what *other* links it would also match, not only by how often it appears on signup forms.

Candidate	Reason
rejected news, free, offers, events	match ordinary menu links (News, Free trial, Offers, Events), which would then be visited before the signup page
rejected daily, latest	match common news-site links such as “Daily crossword” and “Latest stories”
rejected submit, next, save, send, get	as link keywords they match “Next page”, “Submit a tip”, “Save 20%”; kept as submit words only
rejected email, in addresses	appears in contact pages (*/email-us*) and in tracking codes on ordinary links (*?utm_medium=email*)
deferred updates, weekly, stay in the loop	low risk, but not adopted in this round; the first candidates for a follow-up test

6.5 Limits of the keyword study

- The words come from the signup *pages*, while the link keywords are matched against the *links* pointing to them. The two usually share words, but not always.
- The sample is 146 URLs and mostly English; the non-English additions rest on one or two forms each.
- The effect on a crawl has not yet been measured. The planned test is to crawl a set of sites not in the 146 once with the old lists and once with the new ones, and compare how often a form is found and on which page.

---

7. Differences from the original method

Englehardt et al. [1] needed one form per site to submit; we need a complete catalogue of the forms a visitor sees. The main differences:

- Every candidate form is recorded, not only the top-ranked one.
- Bottom-up detection runs on every page, not only when top-down finds nothing.
- Detection runs four times per page (load, delay, scroll, exit intent) so the trigger of each form is known.
- Open shadow DOM is searched.
- Links are opened by navigating to their address, not by clicking, and nothing is ever submitted.
- The paper's “/2017” URL keyword is generalised to any year; the keyword lists were extended as described in section 6, including a new group that matches signup words in link addresses.
- Login detection uses word boundaries, the text around the form, and the page address (section 4).
- robots.txt is not consulted.

- The crawler was moved from Playwright to Selenium on 21 October 2026. The measurement regime is unchanged: on the test fixture both produce identical rows for every form — placement, ranking, field types, text, widths and viewport coverage — provided the viewport is set exactly and scrollbars hidden. Two mechanics differ: the HTTP status is now read inside the page from `PerformanceNavigationTiming`, and audio and video are blocked by address rather than by resource type, so media served from an address with no file extension is now downloaded.

---

8. Limitations

- **Cookie banners** can suppress or delay popups, especially on EU sites. Dismissing them would require clicking, which the method rules out.
- **Bot detection.** Some sites serve different content to automated browsers. The browser window is visible by default for this reason.
- **Not detected:** closed shadow DOM, forms drawn on a canvas, and forms behind CAPTCHAs.
- **Heuristic columns** (placement, popup hint, login-like, consent wording, heading) need checking against the screenshots.
- **Time-outs.** A site stopped by the 6-minute limit keeps the forms found on its earlier pages, but none from the page it was stuck on.
- **Out of scope for the crawler:** what happens after signing up (confirmation emails, double opt-in). Submitting forms is planned as separate work; see section 9.

---

9. Planned work: submitting the forms

The crawler described above never submits anything, and that will not change. Submission is planned as a **separate tool**, run deliberately on a chosen subsample, so that an ordinary crawl can never sign up for anything by accident.

Submitting would extend the study past the design of a form to what the form actually does: the confirmation flow, whether the site uses double opt-in, what the welcome email contains, and where the address ends up afterwards. Section 8 lists this as out of scope for the crawler; a dedicated tool is how it comes into scope.

Four questions need answers before any of it is written:

- **Ethics approval.** Submitting creates real subscriptions, and on some sites real accounts, which detection-only crawling does not. Whether this needs IRB sign-off has to be settled first.
- **Address identity.** One research address, or a per-site alias. An alias per site is what would make it possible to tell later which site sold or leaked an address, which is much of the reason to submit at all.
- **Pacing and scope.** A submission is a write rather than a read: at most one per site, on a small subsample rather than the full list.
- **Records.** What each attempt stores (fields filled, the response, the confirmation page) and whether it joins `forms.csv` or gets a file of its own.

The decisions above are recorded alongside the code so they are settled before implementation rather than during it.

References

1. Englehardt, S., Han, J., & Narayanan, A. (2018). I never signed up for this! Privacy implications of email tracking. *Proceedings on Privacy Enhancing Technologies*, 2018(1), 109–126. Section 3 and Appendix 10.1.